FitU

IN PROGRESS

Fitness and nutrition in one place, built around an interactive calendar.

Project brief by Vel'Shaun Byers

PLANNED STACK React, Tailwind CSS, FastAPI, Python, OAuth 2.0, OpenAI, pytest, GitHub Actions, Vercel, Railway

1. Why I Am Building This

FitU was my capstone project for my coding boot camp. It was the first full stack application I ever built, and it means a lot to me.

I am very big into fitness, and I love creating applications that I use in my personal life. There is not one app I have built in my personal time that I do not use. This is another one. I get a fitness application that tracks my progress, gives me recommendations, and ties my nutrition and my fitness into one place.

It was my original. Now I get to build it again with all the years of experience I have gathered since.

2. What I Hope to Achieve

Success looks like a fully functioning application. Once I have tested it myself for a few months, I plan to put it on the store. I hope it can bring some convenience to other people's lives.

- An interactive calendar is the hub. Click a day and see the workout, nutrition, notes, and weight logged for it.
- An intensity module works out weight and rep adjustments from my baseline lifts, so I do not have to do the math at the gym.
- Workout splits save as templates that fill the calendar by day of the week, and friends can share them.
- Saved meals log with one tap, and daily calorie and macro targets come from my height, weight, activity level, and goal.

3. Integrations

- **Exercise data.** An exercise API that includes how to instructions for each movement. I am evaluating ExerciseDB and wger.
- **Nutrition and recipes.** A recipe and ingredient API so meals can be searched and pinned to a day. I am evaluating Spoonacular.
- **Reminders.** One gentle daily text message if I miss logging a meal, with the option to reply and log it.

Wearable data is an ambition, and I want to be upfront about why it is not there yet. I want sleep, steps, and workouts from Apple Health, Oura, Fitbit, and Whoop to fill the calendar on their own. Apple Health data can only be read through HealthKit from a native iOS app, which requires a paid Apple Developer Program membership [8]. FitU is a web app first, so that integration has to wait for the mobile version. Until then, the data model already has a place for those metrics, and manual entry covers the gap.

4. AI Integrations

- **AI trainer and AI chef.** Two separate personas that the user can name and give a voice. The trainer answers fitness and form questions. The chef handles cooking and nutrition questions.
- **Plans that fill the calendar.** A generated workout plan or meal plan goes straight into the calendar instead of into a document.
- **Photo food logging.** A photo is matched against saved meals first. Nothing is logged until I confirm or correct it, because a pure AI estimate is not accurate enough to log on its own.
- **Reports.** Recommendations that combine nutrition, sleep, and exercise data and explain the why behind them.

Running two personas costs money, so I am using a lighter and cheaper model for them.

5. CI/CD Pipeline

Every push and pull request runs through GitHub Actions [1]. The workflow installs dependencies, runs the test suite, and builds the front end.

The React front end deploys to Vercel and the FastAPI back end deploys to Railway. A change only reaches the live site after the checks pass.

6. Testing

Back end tests are written with pytest [2] and run in the pipeline on every push.

The first tests cover the parts where a wrong number would hurt the most. That means the intensity module math, the calorie and macro target calculations, and the API routes that write to the calendar.

7. Security

- Sign in uses OAuth 2.0 [3] with more than one provider, so FitU never stores a password.
- I use the OWASP Top Ten [4] as my checklist while building and reviewing.
- Body and health data is the data I am most careful with. A user can only read and write their own records.
- A shared workout split only carries the structure, meaning the exercises and days. Personal numbers like weights lifted are never shared.
- API keys and secrets live in environment variables on the host. They are never committed to the repository.

8. Accessibility

I build to the federal standard, Section 508. The Revised 508 Standards incorporate WCAG 2.0 Level AA by reference, so that is the bar I test against [5] [6].

- Every feature works with a keyboard alone, with a visible focus indicator.
- Text and controls meet the Level AA contrast ratios.
- Form fields have labels, and images and icons have text alternatives.
- Information is never carried by color alone.

FitU will ship with preset color themes. Each theme is paired with text colors ahead of time so every theme stays readable.

9. Versioning and Lifecycle

I follow Semantic Versioning [7]. A bug fix bumps the patch number, a new feature bumps the minor number, and a breaking change bumps the major number.

Every release is tagged in GitHub with a short changelog entry. I am the only developer, but the rule does not change. It just means I am the one who enforces it.

The plan is web app first. A mobile app is a later goal, and it is also what unlocks the wearable integrations.

> **Current status.** In progress. FitU is a ground up rebuild of my boot camp capstone. Feature planning comes first, then the build.

References

[1] GitHub. GitHub Actions documentation. https://docs.github.com/en/actions

[2] pytest documentation. https://docs.pytest.org/

[3] Internet Engineering Task Force. RFC 6749, The OAuth 2.0 Authorization Framework. https://datatracker.ietf.org/doc/html/rfc6749

[4] OWASP Foundation. OWASP Top Ten. https://owasp.org/www-project-top-ten/

[5] Section508.gov. Applicability and Conformance Requirements, Revised 508 Standards. https://www.section508.gov/develop/applicability-conformance/

[6] World Wide Web Consortium (W3C). Web Content Accessibility Guidelines (WCAG) 2.0. https://www.w3.org/TR/WCAG20/

[7] Semantic Versioning 2.0.0. https://semver.org/

[8] Apple Developer. HealthKit documentation. https://developer.apple.com/documentation/healthkit